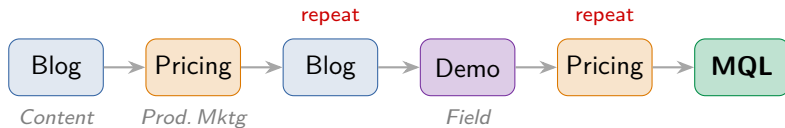


Attribution Modeling

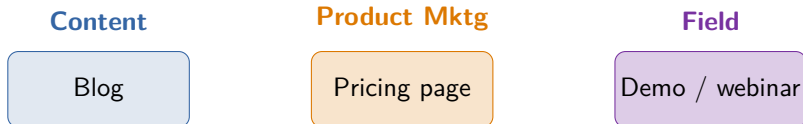
Who gets credit for the conversion? Heuristics · Markov chains · Shapley values ·
Shapley on ML

One prospect's path to becoming an MQL



5 touchpoints, but only **3 distinct channels** — Blog, Pricing, Demo.

Three teams, three assets

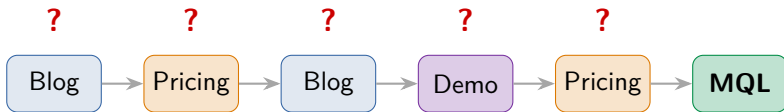


Each team asks the same question:

"Did my asset drive the MQL?"

Budget and headcount ride on the answer.

5 touches, 1 conversion — who gets the credit?



We must split **one** conversion's credit across the channels that led to it.

How much of the MQL does each of Blog, Pricing, Demo deserve?

We see *that* they converted, not *why*

- ▶ The data shows the touches and the outcome — not the cause.
- ▶ A channel being *present* in winning journeys $\neq$ that channel *caused* the win.
- ▶ Popular channels look good simply by appearing everywhere.

So: how do we turn “was present” into “deserves credit”?

Attribution = a share of credit per channel

Goal

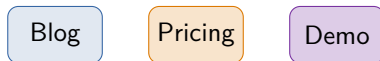
Assign each channel a **credit** number, so the shares are comparable — which assets actually move MQLs.

- ▶ Bigger credit $\Rightarrow$ that channel was more responsible for conversions.
- ▶ Lets each team see the return on its work.

From messy journey to distinct channels



For now, *for simplicity*, we collapse to the *set* of distinct channels ↓



**We just threw away the repeats — Blog and Pricing were each hit twice.
Hold that thought.**

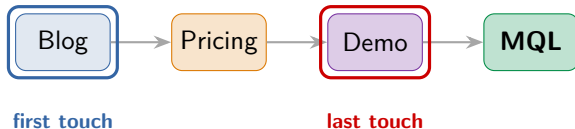
First touch vs last touch

This journey — which single channel deserves the credit?



First touch vs last touch

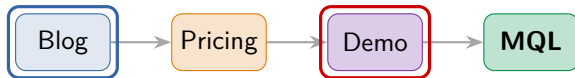
This journey — which single channel deserves the credit?



First-touch credits **Blog**. **Last-touch** credits **Demo**. **Opposite!**

First touch vs last touch

This journey — which single channel deserves the credit?

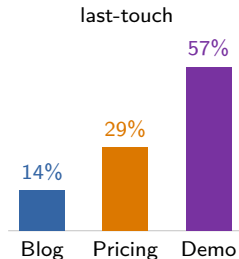
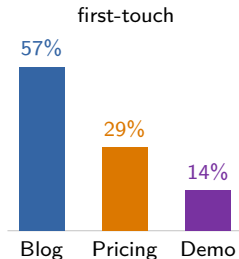


first touch

last touch

First-touch credits **Blog**. **Last-touch** credits **Demo**. **Opposite!**

Apply each rule across *all* journeys $\Rightarrow$ channel-level credit:



Each single-touch rule has a built-in bias

First-touch

- ▶ Over-credits *top-of-funnel* — awareness, discovery.
- ▶ Ignores everything that nurtured and closed.

Last-touch

- ▶ Over-credits *the closer* — the final click.
- ▶ Ignores what built the interest in the first place.

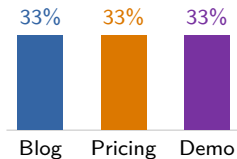
Either way: you reward one touch and erase the rest.

Spreading the credit: multi-touch rules

Give every touch some credit — but how much?
(positions on the journey Blog → Pricing → Demo)

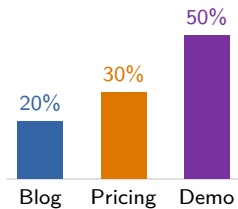
Linear

equal split



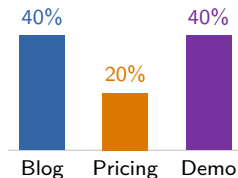
Time-decay

recent weighted



U-shaped

ends weighted



Each is a fixed *position-based* recipe — it never looks at the outcomes.

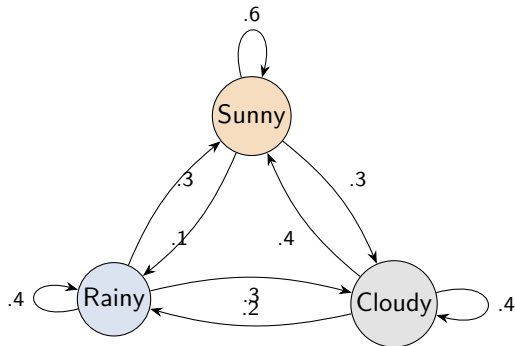
Why 40/40/20, and not 30/30/40?

These splits are **arbitrary** — picked by hand, justified by *nothing in the data*.

- ▶ Every rule so far decides credit by *position*, before looking at outcomes.
- ▶ Change the recipe, change the “winner” — with no principled way to choose.

Data-driven idea: let the *data* decide — which channels actually move conversions?

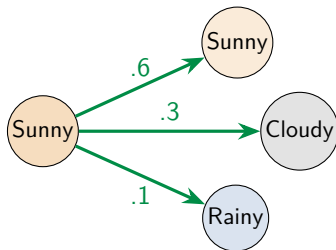
A Markov chain in one picture



States + transition probabilities. Each arrow: “given today, the chance of tomorrow.”

From one state, the chances sum to 1

Given **today is Sunny**, tomorrow is exactly one of:



$$\underbrace{0.6}_{\text{Sunny}} + \underbrace{0.3}_{\text{Cloudy}} + \underbrace{0.1}_{\text{Rainy}} = 1$$

From any state, the outgoing arrows cover every option — they **always sum to 1**.

Memoryless: only *now* matters

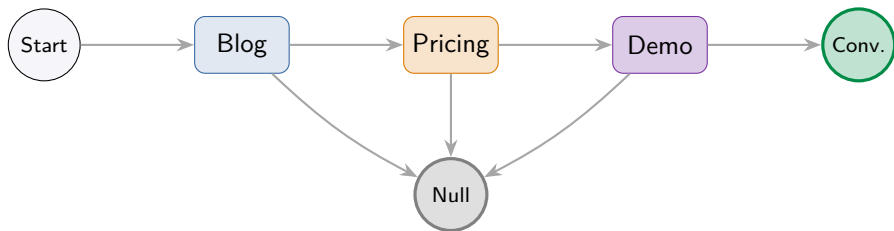
The Markov property

The next state depends **only on the current state** — not on the full history of how you got there.

Weather: tomorrow depends on today, not on what happened last week.

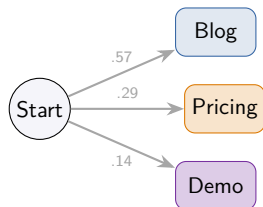
Real journeys *do* have memory — first-order Markov is a deliberate simplification. (Higher-order chains exist; we keep it first-order here.)

A journey is a walk through channel-states



- ▶ States = the channels, plus **Start**, **Convert**, and **Null** (no conversion).
- ▶ **Convert** and **Null** are *absorbing*: once you arrive, you stay.

Estimate transition probabilities by counting

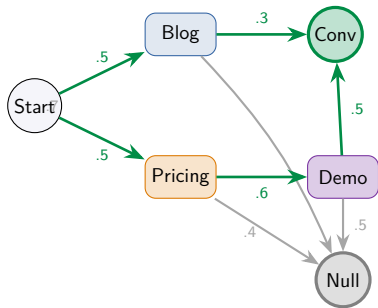


Count transitions; divide by row totals. The **Start** row from our data:

$$\text{Start} \rightarrow (B, P, D) = (.57, .29, .14)$$

Every state's outgoing probabilities sum to 1.

How likely is conversion? Add up the paths



$$P(\text{conv}) = \sum_{\text{Start} \rightarrow \dots \rightarrow \text{Conv}} \prod_{\text{edges}} p$$

Two converting paths:

- St→Blog→Conv: $.5 \times .3 = .15$
- St→Pric→Demo→Conv: $.5 \times .6 \times .5 = .15$

$$P(\text{conv}) = .15 + .15 = \mathbf{0.30}$$

illustrative acyclic graph — green = converting paths

With repeat visits (cycles) the paths are infinite — use the *fundamental matrix*. The script does this for the real numbers.

Remove a channel, watch conversion drop

Removal effect = a channel's importance

$$\text{credit}_i \propto \frac{P(\text{convert}) - P(\text{convert} \mid i \text{ removed})}{P(\text{convert})}$$

- ▶ **Remove a channel** = redirect all its incoming transitions to **Null**.
- ▶ The bigger the drop in conversion, the more that channel mattered.
- ▶ Compute for every channel, then normalize to shares.

Removing Pricing

full chain

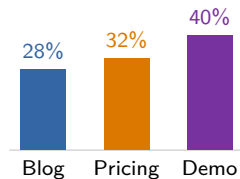
$$P(\text{conv}) = \mathbf{0.386}$$

Pricing $\rightarrow$ Null

$$P(\text{conv}) = \mathbf{0.157}$$

$$\text{removal effect}_{\text{Pricing}} = \frac{0.386 - 0.157}{0.386} \approx 0.59 \quad (\text{a } 59\% \text{ drop})$$

Repeat for all three, normalize $\Rightarrow$ Markov credit shares:



Markov's question

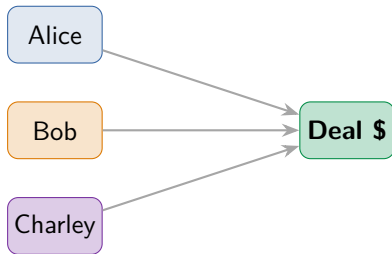
Markov asks:

*“What **breaks** if this channel disappears?”*

A natural “what if we turned it off?” view of importance.

Next: a completely different question — *fairness*.

Three people close one deal — split the bonus fairly

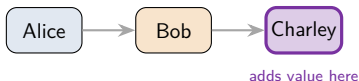


They produced **one** outcome *together*.
What is each person's **fair share** of the bonus?

Channels are the same: several touches jointly produce **one** conversion.

Average marginal contribution over all join orders

- ▶ Imagine the players joining a team *one at a time*.
- ▶ A player's **marginal contribution** = how much value they *add* when they join.
- ▶ **Shapley value** = that contribution, *averaged over every possible order*.



one ordering: Alice, then Bob, then Charley — measure what Charley adds

All 6 orders — Blog's marginal contribution

$v(S)$ = conversion rate of journeys with exactly set S .

order	set before Blog	Blog adds
B, P, D	$\emptyset$	$v(B) - 0 = .10$
B, D, P	$\emptyset$	.10
P, B, D	$\{P\}$	$v(BP) - v(P) = .20$
D, B, P	$\{D\}$	$v(BD) - v(D) = .15$
P, D, B	$\{P, D\}$	$v(BPD) - v(PD) = .15$
D, P, B	$\{P, D\}$	.15

Six “what does Blog add” numbers — one per order.

Average them = Shapley value

$$\phi_{\text{Blog}} = \frac{.10 + .10 + .20 + .15 + .15 + .15}{6} = \frac{0.85}{6} \approx 0.142$$

Same averaging for each channel:

$$\phi_{\text{Blog}} \approx .142, \quad \phi_{\text{Pricing}} \approx .242, \quad \phi_{\text{Demo}} \approx .317$$

$$\underbrace{.142 + .242 + .317}_{\text{the three Shapley values}} = \underbrace{0.70}_{v(\{B,P,D\})}$$

$v(\{B, P, D\})$ = conversion value of all three channels together.

Efficiency: the credits add up to the whole — **no credit lost or invented.**

The same thing, written compactly

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! (|N| - |S| - 1)!}{|N|!} (v(S \cup \{i\}) - v(S))$$

- ▶ The sum runs over every subset S that could be “already there” before i joins.
- ▶ The fraction is just the weight that makes *every join order equally likely*.
- ▶ Exactly the average we did by hand — now in one line.

Why Shapley? It's the *only* fair split

A wishlist any fair credit rule should satisfy:

- ▶ **Efficiency** — the credits add up to the total value.
- ▶ **Symmetry** — equal contributors get equal credit.
- ▶ **Null player** — a channel that adds nothing gets zero.
- ▶ **Additivity** — credit adds up across combined games.

Shapley is the **unique** rule satisfying all four.

Channels as players

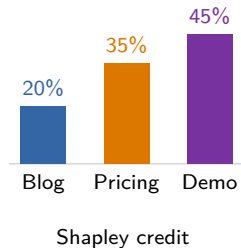
$v(S)$ = conversion rate of journeys whose distinct channel set is **exactly** S .

Predict: which channel earns the most credit?

Channels as players

$v(S)$ = conversion rate of journeys whose distinct channel set is **exactly** S .

Predict: which channel earns the most credit?



First-touch crowned **Blog** (57%). Shapley says Blog earns just **20%** — **Demo** is the real driver.

The catch: 2^n coalitions

- ▶ With n channels there are 2^n subsets to value.
- ▶ 3 channels $\rightarrow$ 8 subsets (easy). 20 channels $\rightarrow$ over a million.
- ▶ Exact Shapley gets expensive fast $\Rightarrow$ in practice, *sample* orderings / approximate.

This cost — and the need for richer features — points to **Shapley on top of ML**.

Raw Shapley sees only presence

- ▶ It uses only *which* channels were present — a 0/1 flag per channel.
- ▶ Rare channel combinations have little or no data $\Rightarrow v(S)$ is noisy or undefined.
- ▶ We want **richer features** and a way to **fill the gaps**.

Idea: let a *model* estimate $v(S)$ instead of raw counts.

Same math, a new value function

§4: empirical

$v(S)$ = conversion rate *counted from data* for set S

§5: model

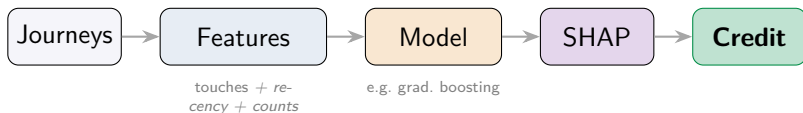
$v(S)$ = the model's *predicted* conversion prob. using only the features in S

Same Shapley formula — the value function goes from *counted-from-data* to *model-predicted*.

Why it helps: the model **interpolates** channel combinations the raw counts never saw.

This is **SHAP** — SHapley Additive exPlanations.

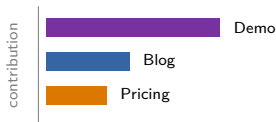
From journeys to SHAP credit



Features can now go *beyond* 0/1 — recency, frequency, dwell, and more.

From one prediction to channel-level credit

- ▶ SHAP explains *one user's* predicted probability — splitting it across that user's features.
- ▶ Average $|\text{SHAP}|$ across all users $\Rightarrow$ aggregate channel credit.



one journey's SHAP contributions to its predicted conversion probability

Everything so far treats a touch as 0/1

Remember the repeated Blog and Pricing visits we dropped on slide 1? That's **frequency** — gone.

Also invisible to these models:

- ▶ **Recency** — how recently each touch happened.
- ▶ **Repetition** — one visit vs ten.

